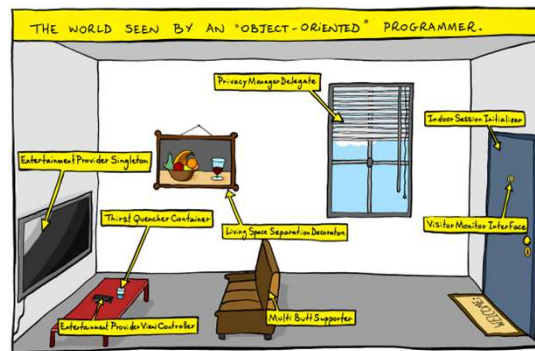




Clean Classes

www.cs.uoi.gr/~zarras/soft-devII.htm

from Clean Code by R. C. Martin, a.k.a "Uncle Bob"

1

Class Organization

Following the standard Java convention, a class should **begin** with a list of **attributes**.

Public static constants, if any, should come **first**. Then **private static attributes**, followed by **private instance attributes**.

Public functions/methods should follow the list of attributes. We like to put the **private utilities** called by a public function right after the public function itself. This follows the **stepdown rule** and helps the program read like a **newspaper article**.

2

Encapsulation

We like to keep our **attributes** and utility functions/methods **private**, but we're not fanatic about it.

Sometimes we need to make an **attribute** or **utility function/methods protected**.

There is **seldom** a good reason to have a **public attribute !!!**

3

Classes should be small!!

Listing 10-1
Too Many Responsibilities

```
public class SuperDashboard extends JFrame implements Metadataliser
{
    public String getCustomiserLanguagePath()
    public void setSystemConfigPath(String systemConfigPath)
    public String getSystemConfigDocument()
    public void setSystemConfigDocument(String systemConfigDocument)
    public boolean getCustomiser()
    public boolean getCustomiserData()
    public boolean getCustomiserData()
    public void showObject(Metadataset object)
    public void showProgress(String s)

    public boolean isMetadataset()
    public void setMetadataset(boolean isMetadataset)
    public Component getMetadatasetComponent()
    public void setMetadatasetComponent(Component lastFocused)
    public void setMetadatasetComponent(boolean isMetadataset)
    public boolean isMetadataset()
    public LanguageManager getLanguageManager()
    public Project getProject()
    public Project getProject()
    public Project getProject()
    public String getProjectName()
    public void setComponentSize(int dimension dim)
    public String getProjectDir()
    public void setProjectDir(String newDir)
    public void updateStatus(int status, int markbox)
    public class() getDashboardClass()
    public MetadatasetReader getMetadatasetReader()
    public void addProject(Project project)
    public boolean setProject(Project project)
    public boolean removeProject(Project project)
    public MetadatasetReader getProjectMetadataset()
    public void resetDashboard()
    public Project loadProject(String filename, String projectName)
    public void setCustomiserMetadataset(boolean customiser)
    public Metadataset getMetadataset()
    public void deleteObject()
    public void setProject(Project project)
    public void editObject(String actionName, ActionEvent event)
    public void setModel(int model)
    public FileManager getFileManager()
    public void setFileManager(FileManager fileManager)
    public ConfigManager getConfigManager()
    public void setConfigManager(ConfigManager configManager)
    public ClassLoader getClassLoader()
    public void setClassLoader(ClassLoader classLoader)
    public Properties getProject()
    public String getSystem()
    public String getSystemDir()
    public int getSystemVersionNumber()
    public int getSystemVersionNumber()
    public int getSystemVersionNumber()
    public Metadataset parsing()
    Metadataset target, Metadataset parent, Metadataset project)
    public void processMetadataset(Metadataset metadataset)
    public void setAppSettings(boolean allowAppSettings)
    public boolean allowAppSettings()
    public boolean allowAppSettings()
    public void setTitle(String title)
    public Metadataset getMetadataset()
    public void showHelp(Metadataset metadataset, String propertyName)
    // ... many non-public methods follow ...
}
```

With **functions** we measured size by counting **physical lines**.

Listing 10-1 outlines a class, **SuperDashboard**, that exposes about **70 public methods**.

Most developers would agree that it's a bit too super in size.

Some developers might refer to SuperDashboard as a **"God class"**

4

Classes should be small!!

Listing 10-2
Small Enough?

```
public class SuperDashboard extends JFrame implements MetadataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

But what if SuperDashboard contained only 5 methods ?

Five methods isn't too much, is it?

5

Classes should be small!!

Listing 10-2
Small Enough?

```
public class SuperDashboard extends JFrame implements MetadataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

With **classes** we use a different measure. We count **responsibilities**.

In this case the **class** is **NOT** **small**, despite its small number of methods, SuperDashboard has too **many responsibilities**.

The **name** of a **class** should describe what **responsibilities** it fulfills.

In fact, **naming** is probably the **first way** of helping determine class size. If we **cannot** derive a **concise name** for a class, then it's likely **too large**. The more **ambiguous** the class name, the more likely it has too **many responsibilities**.

For example, class names including weasel words like **Processor** or **Manager** or **Super** often hint at **unfortunate aggregation of responsibilities**.

6

Classes should be small!!

Listing 10-2

Small Enough?

```
public class SuperDashboard extends JFrame implements MetaDataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

We should also be able to write a **brief description** of the class in about **25 words**, without using the words “if,” “and,” “or,” or “but.”

How would we describe the SuperDashboard?

“The SuperDashboard **provides access to the GUI component** that last held the focus, **and** it also allows us **to track the version and build numbers.**”

The first “and” is a hint that SuperDashboard has too many responsibilities.

7

Classes should be small!!

Listing 10-2

Small Enough?

```
public class SuperDashboard extends JFrame implements MetaDataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

Is there any well known principle that promotes small classes ???

8

The Single Responsibility Principle

Listing 10-2
Small Enough?

```
public class SuperDashboard extends JFrame implements MetaDataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

This principle gives us both a definition of responsibility, and a guideline for class size.

Classes should have one responsibility—one reason to change !!

9

The Single Responsibility Principle

Listing 10-2
Small Enough?

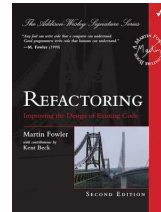
```
public class SuperDashboard extends JFrame implements MetaDataUser {
    public Component getLastFocusedComponent()
    public void setLastFocused(Component lastFocused)
    public int getMajorVersionNumber()
    public int getMinorVersionNumber()
    public int getBuildNumber()
}
```

The seemingly small SuperDashboard class has two reasons to change.

First, it **tracks version information** that would seemingly need to be updated every time the software gets shipped.

Second, it **manages Java Swing components** (it is a derivative of JFrame, the Swing representation of a top-level GUI window).

10



Smells and related refactorings

<https://refactoring.com/catalog/>

11

Large Class

Symptoms

Large number of **instance variables**

Large number of **methods**

Large number of **lines**

Very **general name**

Causes

A little bit at a time. The author adds "**one more capability**" to a class, and eventually it grows too big.

Sometimes the problem is a lack of insight into the parts that make up the whole class.

In any case, the class represents **too many responsibilities** folded into one object.

12

Large Class

What to Do

In general, you're trying to break up the class.

***** If the class has Long Methods, address that smell first.**

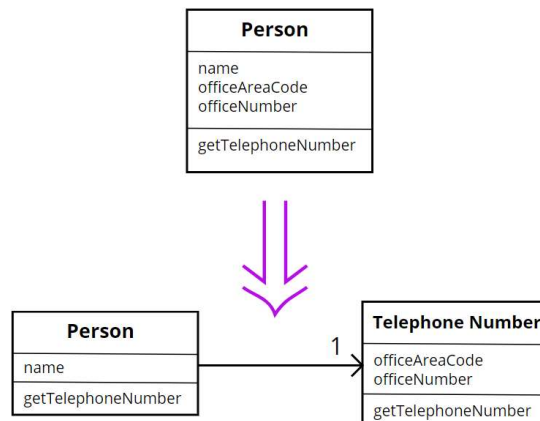
Extract Class, if you can identify a new object that has part of this class's responsibilities

13

Extract Class

You have one class doing work that should be done by two.

Create a new class and move the relevant fields and methods from the old class into the new class.



14

Extract Class

In practice, classes grow.

You add some operations here, a bit of data there. You add a responsibility to a class feeling that it's not worth a separate class, but as that responsibility grows and breeds, the **class becomes too complicated**.

Such a class is one with **many methods** and quite a **lot of data**. A class that is too big to understand easily.

You need to consider where it can be split, and you split it.

A good sign is that **a subset of the data and a subset of the methods seem to go together**. Other good signs are subsets of data that usually **change together** or are particularly dependent on each other.

15

Large Class

What to Do

In general, you're trying to break up the class.

If the class has Long Methods, address that smell first.

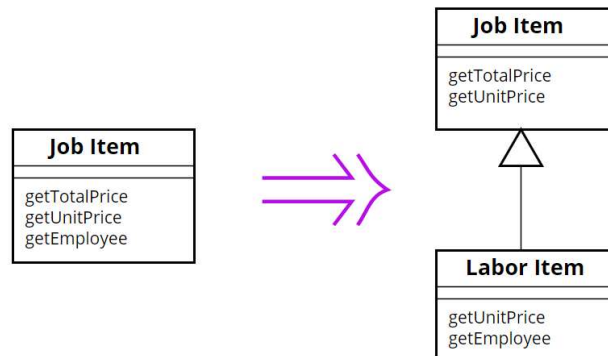
Extract Subclass, if you can divide responsibilities between the class and a new subclass

16

Extract Subclass

A class has features that are used only in some instances.

Create a subclass for that subset of features.



17

Large Class

What to Do

Extract Interface,

you can identify **subsets of features that clients use**

you have no time to extract class or

it is very complex to extract class or

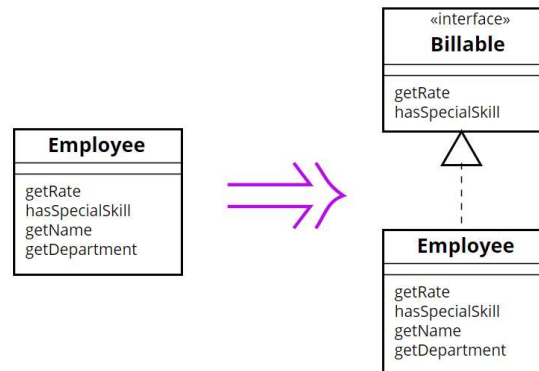
It does not make sense to extract class because it is cohesive

18

Extract Interface

Several clients use the same subset of a class's interface, or two classes have part of their interfaces in common.

Extract the subset into an interface.



19

Extract Interface 21

Classes use each other in several ways.

Use of a class often means ranging over the **whole area of responsibilities** of a class.

Another case is use of **only a particular subset of a class's responsibilities by a group of clients.**

For the second case it is often useful to define an interface for **the subset of responsibilities.**

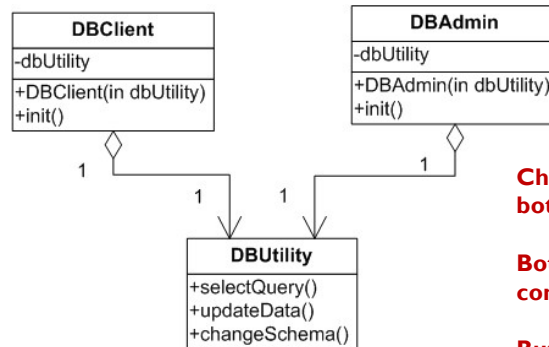
That way it is easier to **see how the responsibilities of a class divide wrt how they are used by other classes and the different roles played by the class**

20

Διαφάνεια 20

- z1** this is a good alternative if you dont have time to extract class and reengineer in general
zarras; 30/11/2018

Extract Interface



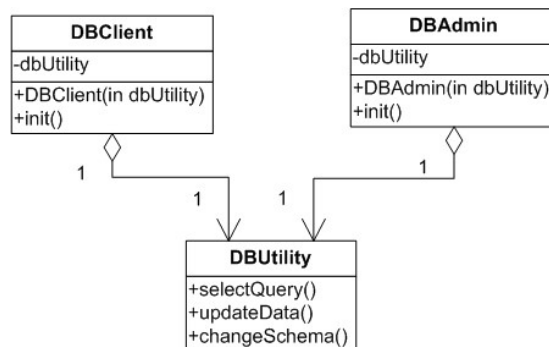
Changes to any method affect both DBAdmin and DBUser

Both should be re-tested, re-compiled, etc...

But it shouldn't because DBUser only uses selectData() while DBAdmin only uses updateData() and changeSchema()

21

Extract Interface



Maybe we should separate selectQuery and updateData, changeSchema to different classes ????

What if we DON'T have time to split DBUtility ?

Or what if it DOES NOT MAKE SENSE to split it because it is generally cohesive, providing a single responsibility (ACCESS TO Database) ?

How can we organize better for change?

22

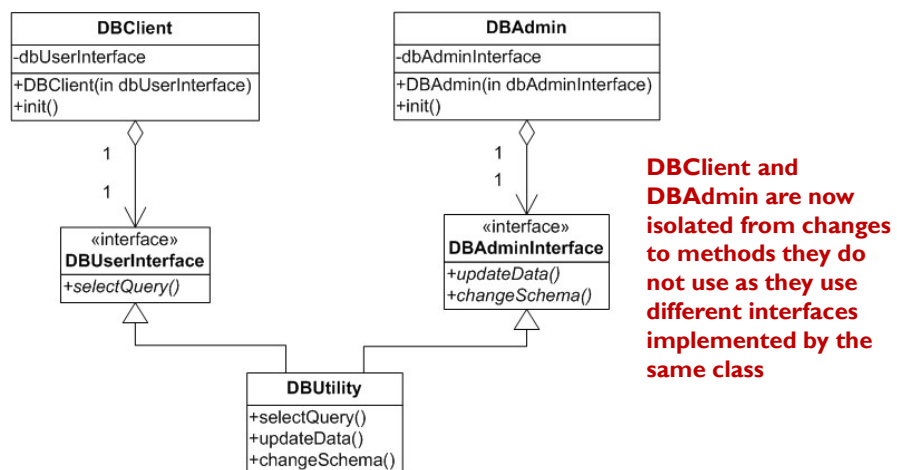
Extract Interface

The **Interface Segregation Principle (ISP)** says that our **classes** should not depend upon methods that they don't use !!

The idea then is to **extract different interfaces** that **match the needs of the client classes...**

23

Extract Interface



24